

GUIÓN DOCENTE — CLASE 1

Arquitectura Flask + Login + Blueprints + Layout SaaS

Duración estimada: 3 horas

Modalidad: Práctica guiada

Proyecto: Dashboard SaaS Café Andino

Nivel: Intermedio — Programación Web + Bases de Datos

OBJETIVO GENERAL DE LA CLASE

Construir y comprender la base arquitectónica del sistema web:

- estructura Flask profesional
 - Blueprints
 - autenticación
 - templates
 - rutas protegidas
 - layout SaaS inicial
-

RESULTADO FINAL DE LA CLASE

Al finalizar la sesión el estudiante tendrá:

- sistema Flask funcionando
 - login funcional
 - dashboard protegido
 - sidebar SaaS visible
 - logout funcionando
-

PRODUCTO VISIBLE QUE DEBES MOSTRAR AL INICIO

Antes de explicar código:

1. Ejecutar sistema
2. Mostrar login
3. Ingresar
4. Mostrar dashboard
5. Logout
6. Intentar entrar sin login

Mensaje inicial

"Esto es lo que construiremos hoy.
Hoy no veremos gráficos ni bases de datos todavía.
Hoy construiremos la estructura profesional del sistema."

ESTRUCTURA DE LA CLASE

Bloque	Tema	Tiempo
1	Introducción y demo	15 min
2	Estructura del proyecto Flask	25 min
3	Configuración y arranque	20 min
4	Blueprints	25 min
5	Formularios Flask-WTF	20 min
6	Login y autenticación	45 min
7	Templates y layout SaaS	35 min
8	CSS y diseño base	20 min
9	Práctica final y cierre	15 min

BLOQUE 1 — INTRODUCCIÓN Y DEMO

Tiempo: 15 min

Qué mostrar

Ejecutar:

```
python run.py
```

Abrir navegador.

Mostrar:

- login
 - dashboard
 - logout
-

Qué decir

“Todo sistema profesional empieza con autenticación y estructura.”

“Hoy construiremos la puerta del sistema.”

“Las gráficas y dashboards vienen después.”

Conceptos clave

- Backend
 - Frontend
 - Flask
 - Arquitectura modular
 - SaaS Dashboard
-

BLOQUE 2 — ESTRUCTURA DEL PROYECTO

Tiempo: 25 min

Archivo a mostrar

Explorador de carpetas completo

Mostrar:

```
project/
|
├─ app/
|   ├── __init__.py
|   ├── config.py
|   └── forms.py
|
|   ├── auth/
|   │   └── routes.py
|
|   ├── dashboard/
|   │   └── routes.py
|
|   ├── templates/
|   │   ├── base.html
|   │   └── login.html
|
|   └── static/
|       ├── css/
|       └── js/
|
├─ run.py
└─ requirements.txt
```

Qué explicar

Idea central

“No es un solo archivo.
Un sistema profesional se divide por responsabilidades.”

Explicar:

run.py

Punto de entrada

app/

Núcleo del sistema

auth/

Todo lo relacionado a login

dashboard/

Vistas protegidas

templates/

HTML dinámico

static/

CSS y JS

Pregunta para estudiantes

“¿Por qué creen que dividir el proyecto así es mejor que hacer todo en un solo archivo?”

BLOQUE 3 — CONFIGURACIÓN Y ARRANQUE

Tiempo: 20 min

Archivo

run.py

Mostrar:

```
from app import create_app

app = create_app()

if __name__ == "__main__":
    app.run(debug=True)
```

Explicar línea por línea

create_app()

Crea la aplicación

debug=True

Modo desarrollo

Archivo

app/init.py

Mostrar:

```
from flask import Flask

def create_app():
```

```
app = Flask(__name__)

return app
```

Conceptos clave

- instancia Flask
- configuración centralizada
- patrón factory

Mensaje docente

“Aquí nace toda la aplicación.”

BLOQUE 4 — BLUEPRINTS

Tiempo: 25 min

Archivo

auth/routes.py

Mostrar:

```
from flask import Blueprint

auth_bp = Blueprint("auth", __name__)
```

Explicar

¿Qué es un Blueprint?

“Es una forma de dividir el sistema en módulos.”

Comparación útil

Sin Blueprint	Con Blueprint
Todo mezclado	Sistema modular
Difícil escalar	Escalable
Caótico	Ordenado

Registrar Blueprint

init.py

Mostrar:

```
from app.auth.routes import auth_bp

app.register_blueprint(auth_bp)
```

Práctica

Agregar nueva ruta simple:

```
@auth_bp.route("/test")
def test():
    return "Hola Flask"
```

BLOQUE 5 — FORMULARIOS Flask-WTF

Tiempo: 20 min

Archivo

forms.py

Mostrar:

```
from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField
from wtforms.validators import DataRequired

class LoginForm(FlaskForm):
    username = StringField("Usuario", validators=[DataRequired()])
    password = PasswordField("Password", validators=[DataRequired()])
```

Explicar

FlaskForm

Formulario seguro

validators

Validaciones automáticas

Práctica

Cambiar:

- labels
- placeholders

Pregunta

“¿Por qué validar desde backend y no solo HTML?”

BLOQUE 6 — LOGIN Y AUTENTICACIÓN

Tiempo: 45 min

Archivo

auth/routes.py

Mostrar login

```
@auth_bp.route("/login", methods=["GET", "POST"])  
def login():
```

Explicar

GET

Mostrar formulario

POST

Procesar datos

Mostrar validación

```
if form.validate_on_submit():
```

Explicar flujo completo

1. Usuario llena formulario
2. Flask recibe POST

3. Se valida
 4. Se autentica
 5. Se crea sesión
 6. Redirección
-

Explicar login_user()

```
login_user(user)
```

Concepto clave

"La sesión queda activa."

Explicar logout

```
logout_user()
```

Explicar rutas protegidas

dashboard/routes.py

```
@login_required
```

DEMO IMPORTANTE

Intentar entrar:

- SIN login
 - CON login
-

Mensaje docente

“Aquí comienza la seguridad del sistema.”

BLOQUE 7 — TEMPLATES Y LAYOUT SaaS

Tiempo: 35 min

Archivo

base.html

Explicar herencia

```
{% block content %}  
{% endblock %}
```

Explicar layout

Sidebar

Navegación

Navbar

Cabecera

Main content

Contenido dinámico

Mostrar estructura SaaS

```
<div class="sidebar"></div>
<div class="main-content"></div>
```

Concepto clave

"No repetimos HTML en cada página."

Práctica

Cambiar:

- nombre sistema
- logo
- menú lateral

BLOQUE 8 — CSS Y DISEÑO BASE

Tiempo: 20 min

Archivo

styles.css

Explicar

Paleta café

```
background: #4E342E;
```

Explicar

Cards

Sidebar

Responsive

Mostrar media query

```
@media(max-width:768px)
```

Práctica

Cambiar:

- color sidebar
 - hover
 - tamaño card
-

BLOQUE 9 — PRÁCTICA FINAL

Tiempo: 15 min

Los estudiantes deben lograr

- ✓ Ejecutar Flask
 - ✓ Login funcional
 - ✓ Logout funcional
 - ✓ Ruta protegida
 - ✓ Sidebar visible
 - ✓ Layout funcionando
-

ACTIVIDAD FINAL

Modificación rápida

Cada estudiante cambia:

- nombre del sistema
 - color principal
 - usuario hardcodeado
-

PREGUNTAS DE CIERRE

- ¿Qué hace Flask-Login?
 - ¿Qué es un Blueprint?
 - ¿Qué archivo crea la app?
 - ¿Qué hace login_required?
-

MENSAJE FINAL DOCENTE

“Hoy construimos la estructura profesional del sistema.”

“En la siguiente clase construiremos la base de datos real usando PostgreSQL y ORM.”

ARCHIVOS QUE DEBES ABRIR EN ORDEN

Orden exacto docente

1. run.py
 2. app/**init**.py
 3. app/config.py
 4. app/auth/routes.py
 5. app/forms.py
 6. app/dashboard/routes.py
 7. templates/base.html
 8. templates/login.html
 9. static/css/styles.css
-

ARCHIVOS QUE NO DEBES TOCAR AÚN

NO mostrar todavía:

- models.py
 - services.py
 - consultas
 - Chart.js
 - DataTables
 - migraciones
-

CONSEJO DOCENTE IMPORTANTE

NO expliques:

- todo Flask
- todo HTML
- todo CSS

Solo:

- flujo
 - arquitectura
 - autenticación
-

RESULTADO PEDAGÓGICO REAL

Esta clase conecta:

- Programación Web
- Arquitectura Web
- Seguridad básica
- Templates
- Diseño moderno

Y deja al estudiante listo para:

- PostgreSQL
- ORM
- dashboards reales